SAPIENZA
UNIVERSITÀ DI ROMA

Advanced Software Engineering

NNModelling

A Visual DSL for Neural Network Design

Authors
Davide Pietragalla 1965270
Luca Sforza 2050030

June 16, 2026

Contents

1 Introduction

NNModelling is a DSL (Domain-Specific Language) for designing neural network architectures using a visual diagrammatic language. Models drawn in the visual editor are compiled into Hydra configuration files, which dynamically instantiate a working PyTorch Lightning model at runtime, enabling training and inference without manual implementation of the architecture.

The project follows a model-driven approach: the DSL syntax was first defined using a UML metamodel, then a front-end application was built to enable visual diagramming, and finally a back-end pipeline was implemented to produce executable configurations.

The system splits into two subsystems: a front-end visual editor built with Svelte 5, Svelte Flow, and TypeScript, where users drag, connect, and configure neural network modules; and a back-end pipeline written in Python using PyTorch Lightning and Hydra/OmegaConf, which converts visual diagrams into Hydra configuration files and dynamically instantiates them into a working PyTorch model.

The core data pipeline is:

Visual Graph → NNTree (AST) → JSON → Pipeline → Hydra YAML → LightningModule

Currently the system supports 27 modules (Linear, Conv2d, ReLU, Tanh, Embedding, LayerNorm, Dropout, various loss functions, etc.), 6 joins (Addition, Concat, Einsum, MatMul, ScaledDotProduct, MaskedScaledDotProduct), and 2 subflow behavioral templates (Repeat for sequential repetition and HorizontalRepeat for parallel copies via `vmap`).

2 Requirements

2.1 Functional Requirements

- **Visual Diagram Editor**: The system shall provide a drag-and-drop canvas where users can place, connect, and configure neural network module nodes using a visual interface.
- **Node Types**: The system shall support distinct node types including modules (single input, single output), input nodes (output only), loss/output nodes, joins (multiple inputs, single output), and subflow containers (collapsible nested graphs).
- **Implicit Forks**: Any node output may connect to multiple downstream nodes without requiring a special fork node.
- **Explicit Joins**: Joins shall be explicit special nodes that accept multiple inputs and produce a single output. Each join has an expression defining the merge operation (addition, concatenation, matrix multiplication, etc.).
- **Stereotype System**: The system shall support stereotype definitions stored as JSON files that define module behavior including Python class name, visual properties (color, dimensions), typed parameters with defaults, and category.
- **Extensible Module Registry**: Adding a new neural network layer shall require only a new JSON stereotype file, with no changes to the editor, compiler, or code generator.
- **Subflow Containers**: The system shall support hierarchical subflow containers that group related nodes into reusable sub-architectures.
- **Recursive Nesting**: Subflows shall be nestable inside other subflows to any depth.

- **Behavioral Stereotypes**: Subflows shall support behavioral stereotypes including `Repeat` (duplicates internal subgraph N times in sequence) and `HorizontalRepeat` (creates N parallel copies with independent weights executed via `vmap`).
- **Subflow Persistence**: Users shall be able to save and load individual subflows to and from disk independently.
- **Parameter Configuration**: Each node shall support typed parameters (int, float, bool, list, dict, None) configurable through the visual editor.
- **Graph Compilation**: The system shall compile the flat visual graph into a hierarchical tree representation (NNTree) using topological sorting.
- **Code Generation**: The system shall generate executable Hydra configuration files from the compiled tree representation.
- **Dynamic Model Instantiation**: The back-end shall dynamically build a PyTorch Lightning module from the generated configuration at runtime.
- **Save/Load Diagrams**: Users shall be able to save diagrams as JSON files and reload them for editing.
- **Training Pipeline**: The system shall support training the generated model with configurable optimizer, trainer, dataset, and logging parameters.
- **Inference**: The system shall support inference on trained models with output export including predictions JSON and image montages.

2.2 Non-Functional Requirements

- **Extensibility**: The stereotype-driven architecture shall allow adding new module types via JSON files without any code modification to the editor or back-end pipeline.
- **Platform Independence**: The front-end shall run in any modern web browser with no server-side dependencies.
- **Separation of Concerns**: The front-end shall handle only DSL logic (visual editing, graph compilation, stereotype management). The back-end shall handle only network instantiation, training, and inference. The two subsystems communicate via a well-defined JSON interchange format.
- **Test Coverage**: The system shall maintain comprehensive test coverage across both front-end and back-end components.
- **Expressiveness**: The DSL shall be capable of expressing modern neural network architectures including transformers with multi-head attention, residual connections, and sequential repetition.
- **Execution Correctness**: The forward pass shall use BFS topological traversal with in-degree tracking to guarantee correct execution order through complex fork-join topologies.

3 Design

3.1 UML Metamodel

The DSL syntax was formally defined using a UML metamodel designed in Visual Paradigm. The metamodel captures the core abstractions of the language and their relationships. Three focused views are presented below.

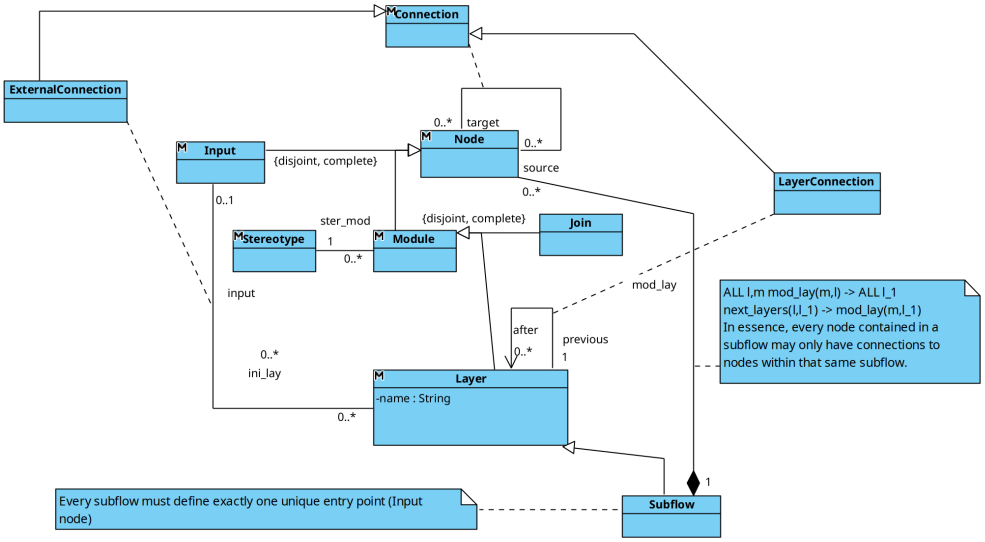

Figure 1: UML metamodel view showing the node type hierarchy: Module, Layer, Subflow, Input, and their relationships.

The first view (Figure 1) illustrates the node type hierarchy. `Node` is an abstract base class with concrete subtypes including `Module` (standard neural network layers such as Linear and Conv2d), `Layer`, `Input`, `Join`, `Subflow`, `ExternalConnection`, and `LayerConnection`. The composition relationship `Subflow "1" --> "0..*" Node : mod_lay` enables recursive containment — a Subflow may contain any Node type, including other Subflows. The `Input "0..1" --> "0..*" Layer : ini_lay` association allows optional input connections to layers for initialization dependencies.

The second view (Figure 2) models the connection system. Each `Node` is associated with zero or more `Handle` instances via the `nod_han` relationship. `SourceHandle` and `TargetHandle` specialize `Handle`. A `Connection` links a `SourceHandle` to a `TargetHandle`, representing a directed edge in the visual graph. This design supports the implicit fork model: a SourceHandle may participate in multiple Connections, enabling one-to-many fan-out, while each TargetHandle accepts exactly one Connection, enforcing single-input semantics.

The third view (Figure 3) captures the stereotype system. A `Stereotype` defines a module template and is associated with zero or more `Parameters` via `par_ster`. A `StereotypeApplication` links a Stereotype to a Module instance (`ster_mod`) and carries concrete `ParameterInstance` values (`ster_par_istance`). Each ParameterInstance references a Parameters definition (`istance_par`). This separation between definition (Stereotype + Parameters) and instance (StereotypeApplication + ParameterInstance) allows the same module type to be instantiated with different parameter values across multiple nodes.

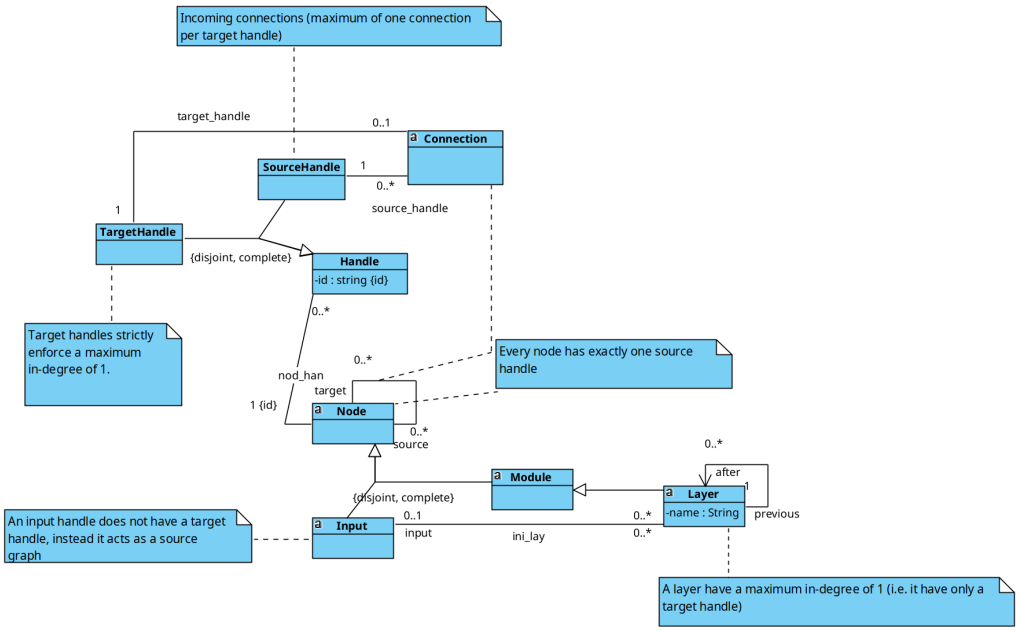

Figure 2: UML metamodel view showing the handle and connection system: SourceHandle, TargetHandle, Connection, and their association with Node.

3.2 DSL Concepts

3.2.1 Nodes and Edges

The DSL operates on a directed graph where nodes represent computational units and edges represent data flow. There are four main node categories:

- **Module**: The fundamental building block. Each module receives exactly one input connection and can output to any number of downstream nodes. Every module is associated with a stereotype that defines its operation (e.g., `nn.Linear`, `nn.Conv2d`), visual properties, and configurable parameters.
- **Input**: A special node with no input handle and only outgoing connections. The input node defines the entry point of the data flow graph. It is auto-spawned when a new diagram is created.
- **Loss/Output**: Output nodes representing loss functions (e.g., CrossEntropyLoss, MSELoss) or undefined outputs. They have no output handle and determine the model's task type for metric selection.
- **Join**: A special multi-input node that accepts N incoming connections and produces a single output. Joins are explicit because modules accept only one input. Input ordering is preserved via `targetHandle` attributes on edges (e.g., `in-0`, `in-1`), which is critical for non-commutative operations.

3.2.2 Stereotype System

The stereotype system is the backbone of the DSL's extensibility. Stereotypes are stored as JSON files organized by category (`Modules/`, `Joins/`, `SubFlows/`). Each JSON file defines:

- `pythonClassName`: The Python class for code generation (e.g., `nn.Linear` resolves to `torch.nn.Linear`).

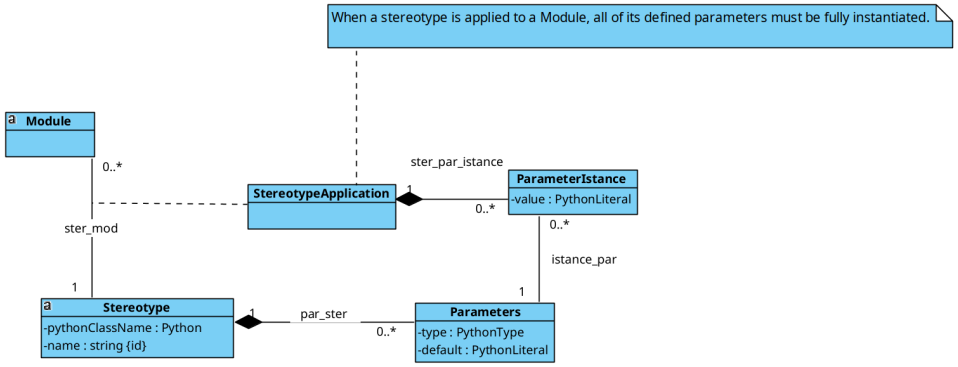

Figure 3: UML metamodel view showing the stereotype system: Stereotype, Parameters, StereotypeApplication, and ParameterInstance.

- `view`: Visual properties including color and dimensions for the canvas rendering.
- `params`: Typed parameters with defaults, types, and display positions on the node.
- `category`: Determines node behavior — standard module, join, input, loss, or subflow.

Stereotypes are loaded at compile time via Vite's `import.meta.glob`, scanning the `Stereotypes/**/*.json` directory structure automatically. This design ensures that adding a new neural network layer requires only a new JSON file — no changes to the editor, compiler, or code generator. Currently 35 stereotypes are defined across the three categories.

3.2.3 Subflow Pattern

Subflows are collapsible containers that group related nodes into reusable sub-architectures. Key characteristics:

- Subflows can be nested recursively (subflows inside subflows).
- Internally, each subflow uses BFS topological traversal with in-degree counting for correct execution order.
- Collapsed subflows hide their internal nodes visually, but the compiler still processes them — collapsed subflow internals are not filtered from compilation.
- The `Fork` stereotype enables passthrough nodes for explicit fork points, enabling skip connections inside subflows (e.g., residual blocks with Repeat).
- Behavioral stereotypes extend subflow semantics (see below).

Behavioral Stereotypes Two behavioral stereotype templates ship with the system:

- **Repeat**: Duplicates the internal subgraph N times in sequence, similar to `nn.Sequential`. The number of repetitions is controlled by the `iterations` parameter. Each copy maintains independent weights.
- **HorizontalRepeat**: Creates N parallel copies of the subgraph, executes them via `vmap` with independent weights using `functional.call` and `stack_module_state`, and concatenates outputs along the last dimension. The number of copies is controlled by the `n` parameter. The join operation is currently hardcoded to concatenation on `dim=-1`.

3.3 Pipeline Overview

The data pipeline transforms visual diagrams into executable models through these stages:

$$\text{Visual Graph} \xrightarrow{\text{NNTree}} \text{JSON} \xrightarrow{\text{Pipeline}} \text{Hydra YAML} \xrightarrow{\text{Hydra}} \text{LightningModule}$$

4 Implementation

The system is divided into two independent subsystems. The front-end contains all DSL logic (visual editing, graph compilation, stereotype management) and runs entirely in the browser. The back-end consumes the compiled JSON output and instantiates the corresponding PyTorch model. Currently, communication between the two subsystems occurs via file exchange (JSON export/import).

4.1 Front-End: Visual DSL Editor

The front-end is built with Svelte 5, Svelte Flow (`@xyflow/svelte`), and TypeScript. It runs entirely in the browser with no server-side dependencies.

4.1.1 Core Architecture

The central state manager is the `Diagram` class (Svelte 5 `$state.raw` reactive arrays), which holds all nodes and edges. It provides methods for adding modules, joins, and subflows; deleting nodes; importing and exporting JSON; and toggling subflow collapse state. All state is reactive — changes to the diagram automatically propagate to the UI.

Stereotypes are loaded via Vite's `import.meta.glob` from the `Stereotypes/**/*.json` directory structure. The `Stereotype` class (stereotype.ts) provides helper accessors (`isJoin`, `isInput`, `isLoss`, `isSubFlow`) derived from each stereotype's `category` field, enabling type-safe node behavior without hardcoded type checks.

4.1.2 Node Rendering

Three Svelte components render different node types on the canvas:

- `CustomNode.svelte`: Standard module nodes with a single input handle on top and single output handle on bottom. Visual appearance (color, size) is driven by the stereotype's `view` properties. Parameters are displayed according to their declared positions (top, bottom, or inline).
- `JoinNode.svelte`: Multi-input merge nodes with N input handles (dynamically generated based on incoming edge count) and a single output handle. The node label shows the join operation type.
- `SubflowNode.svelte`: Collapsible container nodes with a colored header bar. When collapsed, internal nodes are hidden but still participate in compilation. When expanded, internal nodes are rendered as children of the subflow group.

4.1.3 Graph Compilation (NNTree)

The `NNTree` class (conversion/nnTree.ts) converts the flat visual graph into a hierarchical tree representation using Kahn's topological sort. It produces four node types:

- `sequential`: Consecutive single-input–single-output nodes compressed into a linear chain.

- `module`: Individual module nodes with no fan-out merging, preserving their stereotype and parameters.
- `join`: Merge nodes with multiple ordered inputs. Input ordering is preserved from edge `targetHandle` attributes.
- `subflow`: Container nodes with an `entryNode` reference and an internal nodes map preserving full topology. Compiled recursively via `compileSubflowGraph`. Nested subflows are supported through this recursive mechanism.

Subflow compilation includes three phases: first, boundary detection identifies which nodes belong to which subflow based on parent-child relationships; second, Kahn's topological sort orders the internal graph; third, Repeat stereotype unrolling expands the subgraph.

4.1.4 Connection Validation

The `checkValidConnection` function enforces connection rules: each target handle accepts only one connection; source handles allow unlimited outgoing connections (implicit forks); ancestry loop detection prevents circular parent references; and nodes dragged into subflow containers are auto-reparented.

4.2 Back-End: Network Instantiation

The back-end is written in Python and is responsible for consuming the compiled NNTree JSON, generating Hydra configuration files, and dynamically instantiating a working PyTorch Lightning model. It uses PyTorch for tensor operations, PyTorch Lightning for training infrastructure (LightningModule, Trainer), and Hydra/OmegaConf for configuration management.

4.2.1 Configuration Generation

The pipeline script reads the NNTree JSON and generates seven YAML configuration files:

- `net/`: Network architecture with node definitions, layer targets, and parameter values.
- `optimizer/`: Optimizer configuration (Adam, SGD, etc.) with learning rate and hyperparameters.
- `trainer/`: Lightning Trainer settings (max epochs, accelerator, precision).
- `dataset/`: Dataset class selection and parameters.
- `wandb/`: Weights & Biases logging configuration.
- `early_stopping/`: Early stopping criteria and patience.

Key processing steps include: parsing parameter strings via `ast.literal_eval` (supporting int, float, bool, list, dict, and None types); resolving `pythonClassName` to Hydra `_target_` paths (e.g., `nn.Linear` → `torch.nn.Linear`); using `_recursive_: False` on subflow nodes to prevent Hydra from recursing into internal node maps; detecting task type (classification/regression) from loss function selection for automatic metric configuration; and supporting `--num-classes` and `--dataset` CLI flags for configurable output dimensions and dataset selection.

4.2.2 Dynamic Model Execution

The `Net` class, a `LightningModule`, dynamically builds its architecture from the Hydra configuration at runtime:

- A `ModuleDict` is populated by dispatching on node type: `sequential` chains are wrapped in `nn.Sequential`; `module` nodes are instantiated individually via Hydra's `instantiate`; `join` nodes reference custom operation classes; and `subflow` nodes reference Subflow, Repeat, or HorizontalRepeat operation classes.
- The forward pass uses BFS traversal with in-degree tracking to ensure correct execution order through complex fork-join topologies.
- Join nodes collect all inputs ordered by `targetHandle` before executing the merge operation.
- Loss functions are excluded from the execution graph and stored separately for metric selection.
- Flatten is explicit via the Flatten stereotype — no auto-flatten heuristic.

4.2.3 Custom Operations

Eleven custom Python operations are implemented in the back-end, spanning three categories:

- **Join operations (6)**: Addition, Concat, Einsum, MatMul, ScaledDotProduct, and MaskedScaledDotProduct handle multi-input merges. Each inherits from a common base and implements a `forward` method that receives an ordered list of input tensors.
- **Subflow operations (3)**: Subflow implements BFS internal graph traversal for arbitrary subgraphs; Repeat wraps N sequential copies in `nn.Sequential`; HorizontalRepeat uses `vmap` with `functional.call` and `stack_module_state` for N parallel copies. HorizontalRepeat's join is currently hardcoded to concatenation on the last dimension.
- **Module operations (2)**: PositionalEncoding computes sinusoidal positional encodings; SequencePool computes mean pooling over the sequence dimension.

5 Validation

5.1 Test Suite

The project includes 179 automated tests across both front-end and back-end:

- **Front-end (76 tests, Vitest)**: Covers NNTree compilation including sequential chains, skip connections, joins, subflow boundary mapping, Repeat unrolling (3 copies), nested subflows, collapsed nodes, Fork passthrough, and error handling (empty graph, cycles, missing stereotypes). An additional 10 tests cover connection validation rules.
- **Custom operations (38 tests, pytest)**: Unit tests for all 11 custom Python operations, covering forward pass correctness, tensor shape validation, and edge cases.
- **Config generation (36 tests, pytest)**: Parameter parsing via `ast.literal_eval`, layer config building, nested subflow config with `_recursive_: False`, and YAML structure verification across 5 diagram fixtures.

- **Dynamic model (21 tests, pytest)**: In-degree computation, Net initialization, ModuleDict population, BFS forward traversal for various topologies (sequential, fork-join, nested subflows).
- **Integration (8 tests, pytest)**: End-to-end pipeline from JSON fixture through Hydra configuration to Net instantiation and forward pass with real tensor shapes and gradient flow checks.

5.2 DSL Expressiveness Validation

To validate that NNModelling can express modern architectures, a full transformer classifier was built entirely within the visual DSL. The model includes:

- Token embedding layer (30,522 vocabulary $\times$ 128 dimensions).
- Sinusoidal positional encoding.
- A 2-layer TransformerEncoder constructed via the `Repeat` stereotype (`iterations=2`).
- Multi-head attention with 4 heads, implemented via `HorizontalRepeat` (`n=4`) wrapping a single-head attention subflow with Q/K/V linear projections.
- Residual connections via explicit `Fork` nodes and `Addition` joins.
- Sequence pooling via `SequencePool` (mean over sequence dimension).
- Linear classifier head with 2 output classes.

The diagram compiled successfully to a working PyTorch model that was trained on the EnronSpam text classification dataset. Training metrics confirmed correct gradient flow and convergence, validating the DSL's expressiveness for feedforward, convolutional, and encoder-only transformer architectures.

6 Conclusions

6.1 Achievements

NNModelling demonstrates that a visual DSL can effectively bridge the gap between neural network architecture design and implementation. The system successfully compiles complex architectures including transformers with multi-head attention, residual connections, nested subflows, and sequential repetition.

Key achievements include:

- A full end-to-end pipeline from visual design to trained model.
- An extensible stereotype system requiring no code changes for new layer types — 27 modules, 6 joins, and 2 subflow templates are currently supported.
- Support for complex architectural patterns: subflows with recursive nesting, residual connections, parallel branches via `vmap`-based HorizontalRepeat, and sequential repetition via Repeat.
- Comprehensive test coverage (179 tests) across both front-end and back-end, ensuring reliability of the compilation and code generation pipeline.

- A model-driven approach with a formal UML metamodel, ensuring the DSL syntax is well-defined and verifiable.
- Clean separation of concerns: the front-end handles all DSL logic (visual editing, graph compilation, stereotype management), while the back-end handles network instantiation, training, and inference. This decoupling enables independent development and testing of each subsystem.
- A dynamic model instantiation system that builds PyTorch Lightning modules from configuration at runtime, enabling rapid experimentation without code changes.

6.2 Limitations

Several limitations remain. Training loop strategies such as GANs, contrastive learning, and adversarial training cannot be expressed in the DSL and must be implemented in Python outside the framework. The HorizontalRepeat join type is hardcoded to concatenation on the last dimension and is not user-configurable. The forward pass executes nodes sequentially in a single-threaded Python queue loop, even when the DAG contains independent fork-join branches that could run concurrently on separate CUDA streams — though HorizontalRepeat's `vmap`-based execution is an exception, vectorizing N independent copies into a single batched GPU kernel and achieving true parallel execution at the subflow level. Additionally, there is no support for pre-trained models or transfer learning workflows.

6.3 Future Work

Several directions for future development are planned:

- **Network Communication**: The current file-based exchange between front-end and back-end will be replaced with a network communication layer. A training server API is planned to enable one-click training directly from the visual editor, with the front-end sending diagrams to the back-end over HTTP or WebSocket. This will eliminate the manual file export–import workflow and enable real-time training progress monitoring from the editor.
- **Configurable Join Type**: The HorizontalRepeat's hardcoded concatenation join will be replaced with a configurable `join_type` parameter supporting sum, mean, product, or custom aggregation strategies.
- **Parallel Graph Execution**: The single-threaded topological iteration will be replaced with a concurrent CUDA stream dispatcher for arbitrary fork-join topologies, enabling true parallel execution of independent branches on separate GPU streams.
- **Interactive Deployment**: HuggingFace Spaces deployment is planned for interactive browser demos, allowing users to test NNModelling without local installation.
- **Pre-trained Model Support**: Integration of transfer learning workflows with support for loading pre-trained weights into DSL-defined architectures, enabling fine-tuning scenarios.

References

[1] Svelte 5 Documentation. `https://svelte.dev/docs`

[2] Svelte Flow Documentation. `https://svelteflow.dev/`

[3] Paszke, A. et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” NeurIPS 2019.

[4] Falcon, W. et al. “PyTorch Lightning.” `https://lightning.ai/`

[5] Vaswani, A. et al. “Attention Is All You Need.” NeurIPS 2017.

[6] Yadan, O. “Hydra - A Framework for Elegantly Configuring Complex Applications.” `https://hydra.cc/`